

UNIVERSIDAD ANÁHUAC PUEBLA

Inteligencia Artificial

Dr. Paulo Daniel Vázquez Mora

Proyecto Agentes — Fase 3

SIGMA-Vial

Adaptación del sistema a una arquitectura multiagente con CrewAI

Integrante	Rol	No. de ID
Gonzalo Novelo Suchil	Líder de arquitectura	00621893
Emilio Cue Funes	Ingeniero de conocimiento / ontologías	00610400
Amairany Fernández Prieto	Investigador — estado del arte	00602972
Alejandro Victoria Castelan	Desarrollador — factibilidad técnica	00614180
Leticia Tamariz Hernández	Documentador / editor	00373898

Orión Norte, La Vista S/N, Country Club, 72810. San Andrés Cholula, Pue.

Noviembre de 2026

Resumen ejecutivo

En esta fase convertimos SIGMA-Vial en un **equipo de agentes de inteligencia artificial** que trabajan juntos, usando la librería **CrewAI**. Un «agente» es un programa cuyo razonamiento lo hace un modelo de lenguaje (LLM, como el que está detrás de ChatGPT) y que puede usar herramientas para consultar datos y hacer cálculos.

Antes de usar agentes había que responder una pregunta: ¿qué parte del problema les toca? La respuesta no es «todo». Un semáforo debe decidir cada 5 segundos y sus reglas de seguridad deben cumplirse siempre, sin excepción; un modelo de lenguaje tarda segundos o minutos en responder y no es completamente predecible, así que **no puede manejar el semáforo**. Por eso el sistema se divide en dos capas: la **capa de tiempo real** (definida en las Fases 1-2, aún no implementada aquí) decide qué hace el semáforo en cada momento; la **capa de supervisión** (esta fase) corre aparte, una vez por jornada: revisa cómo se comportó el sistema, propone ajustes de configuración, verifica que cumplan la normativa y publica la versión aprobada.

El equipo tiene cuatro agentes (**Analista**, **Ingeniero**, **Auditor** y **Sintetizador**) que ejecutan seis tareas en orden. Todo lo que se puede calcular o verificar lo hacen herramientas escritas en Python tradicional; el modelo de lenguaje solo decide cuál usar e interpreta sus resultados.

La ejecución de demostración usa un modelo de lenguaje real que corre en la computadora del equipo (`qwen2.5:7b`, mediante el programa Ollama) y se hizo desde el notebook en 3 minutos. El equipo publicó una configuración que cumple todas las reglas de seguridad. Antes se probó el sistema muchas veces con distintas versiones; todas se reportan en la sección D.9, incluidas las que fallaron.

El hallazgo principal de la implementación es de diseño: **un modelo pequeño no debe decidir nada que se pueda verificar con código.** Cada falla que apareció (propuestas con tiempos de semáforo inválidos, una configuración incompleta que pasó la revisión, un reporte que inventaba costos y archivos, un modelo que se quedó «pensando» sin parar) se resolvió pasando esa decisión a código determinista: reglas de seguridad, una calculadora de planes, una revisión automática de las respuestas y una verificación del reporte. El valor de los agentes está en interpretar los datos, ponderar prioridades y explicar los cambios.

Límites que conviene tener presentes al leer este documento

- Los datos son **sintéticos** (inventados con una semilla fija) y el problema de la intersección I3 fue **sembrado a propósito**; la ejecución demuestra que la arquitectura funciona, no que SIGMA-Vial mejore el circuito real.
- Las propuestas se revisan contra las **reglas**, pero no se simula cómo se comportarían en la calle: falta conectar SUMO.
- Un modelo local de 7 mil millones de parámetros es **inconsistente**: entre ejecuciones iguales el resultado cambia, y su texto puede contener errores. Por eso lo importante lo impone el código.
- Al analizar las reglas apareció un resultado sobre la especificación de las Fases 1-2: con las esperas máximas de 60 s, el ciclo máximo posible es de unos 82 s, no de 120 s (sección B.5).

Glosario rápido

Término	Qué significa en este documento
LLM / modelo de lenguaje	Programa de inteligencia artificial que lee y escribe texto (por ejemplo, ChatGPT). Aquí es el «cerebro» de cada agente.
Agente	Un LLM con un rol, una meta y un conjunto de herramientas.
Crew (equipo)	El conjunto de agentes y tareas que CrewAI ejecuta en orden.
Herramienta (tool)	Función de Python tradicional que el agente puede invocar: leer datos, calcular, validar. No usa IA, siempre da el mismo resultado.
Regla dura (R0 a R6)	Condición de seguridad que ninguna propuesta puede violar (por ejemplo, tiempo mínimo para que un peatón cruce).
Guardrail	Comprobación automática que se aplica a la respuesta de un agente antes de aceptarla.
Stub	Sustituto de prueba del modelo de lenguaje que responde con un guion fijo. Sirve para probar el código, no demuestra razonamiento.
Spillback	Cuando la fila de vehículos de un cruce llega hasta el cruce anterior y lo bloquea.
Wilcoxon	Prueba estadística que compara dos conjuntos de mediciones y dice si la diferencia es real o ruido ($p < 0.05$ indica diferencia real).
Telemetría sintética	Datos de tráfico generados por un programa, en lugar de medidos en el circuito real.
Ollama	Programa que permite ejecutar modelos de lenguaje en la propia computadora, sin enviar datos a internet.

Bloque A. Justificación de la arquitectura MAS y elección de CrewAI

En palabras simples

Un sistema multiagente (MAS) es un equipo de programas, cada uno con un trabajo distinto, que se pasan resultados entre sí. Aquí el equipo revisa cómo está funcionando el sistema de semáforos, propone ajustes y verifica que no rompan ninguna regla de seguridad. Este bloque explica por qué un equipo es mejor que un solo programa, y por qué elegimos la librería CrewAI para armarlo.

A.1 Motivación del rediseño

El punto de partida: por qué un LLM no puede controlar un semáforo

Antes de justificar la arquitectura multiagente conviene delimitar qué parte del problema admite agentes basados en modelos de lenguaje, porque la respuesta no es «todo».

CrewAI orquesta agentes cuyo motor de razonamiento es un LLM. Cada paso implica al menos una llamada al modelo, con latencias de segundos a minutos, salidas en lenguaje natural y comportamiento no reproducible entre corridas. La ficha de las Fases 1-2 comprometió, en cambio, un Controlador que decide cada 5 s, un verde mínimo de 8 s, un ámbar de 3 s y un Supervisor que garantiza cero violaciones de reglas duras.

Insertar un LLM en ese lazo sería incorrecto por tres razones independientes. Primero, no cumple el tiempo de respuesta. Segundo, destruye la verificabilidad: una regla dura evaluada por un modelo probabilístico deja de ser una regla dura. Tercero, es desproporcionado: una evaluación de utilidad que un cálculo numérico resuelve en microsegundos no justifica una llamada a un modelo de miles de millones de parámetros.

La conclusión no es que el enfoque MAS con CrewAI no aplique, sino que aplica en otra capa del problema.

Los dos regímenes temporales del problema

	Capa de tiempo real (Fases 1 y 2)	Capa MAS de supervisión (Fase 3)
Qué decide	Qué hace el semáforo **ahora**: extender el verde, avanzar de fase, servir al peatón.	Cómo debe **comportarse** el sistema: con qué pesos, qué planes, qué umbrales.
Cadencia	Cada 5 s, o en cada cambio de fase.	Por jornada simulada, o cuando el desempeño se degrada.
Naturaleza	Determinista, acotada en tiempo, verificable.	Interpretativa, abierta, con criterios en conflicto.
Tecnología	Python sobre SUMO/TraCI; utilidad calculada y reglas duras codificadas.	CrewAI con agentes LLM y herramientas deterministas.
Agentes	Monitor, Controlador, Supervisor (Fases 1-2).	Analista, Ingeniero, Auditor, Sintetizador (esta fase).
Estado	Especificada; **no implementada** en este repositorio.	Implementada y ejecutada (Bloque D).

Lo que la capa MAS aporta no es velocidad sino juicio: interpretar por qué una intersección se degradó, cruzar ese diagnóstico con la normativa escrita, ponderar un objetivo de flujo contra uno de equidad peatonal y redactar la justificación de un cambio son tareas abiertas, sin una función objetivo única, donde la latencia es irrelevante.

Por qué multiagente y no un agente monolítico

Un único agente con las seis herramientas y un prompt largo que lo instruyera a diagnosticar, proponer, auditar y redactar tendría cuatro problemas de diseño. No se probó empíricamente una versión monolítica; el argumento es de diseño y se apoya en lo observado en las corridas de prueba (sección D.9).

1. **Conflicto de incentivos dentro de un mismo contexto.** Quien propone un ajuste y quien lo audita persiguen objetivos opuestos: uno maximiza desempeño, el otro impide excepciones. En un solo contexto tienden a contaminarse. La literatura reporta que los LLM no corrigen de forma fiable su propio razonamiento sin retroalimentación externa (Huang et al., 2024); por eso la auditoría la hace un rol distinto, apoyado en un validador determinista.
2. **Dilución del espacio de herramientas.** Con seis herramientas disponibles en todo momento crece la probabilidad de invocar la incorrecta. Acotando cada agente a dos o tres, el conjunto de acciones válidas en cada paso se reduce.
3. **Imposibilidad de atribuir responsabilidad.** Si un diagnóstico resulta equivocado, en un agente monolítico no hay forma de saber en qué etapa se introdujo el error. Con tareas separadas, cada salida intermedia queda registrada y atribuida (en este proyecto, además, guardada en disco).
4. **Contexto largo.** El flujo completo acumula telemetría, normativa, topología, calendario y bitácora. En tareas separadas cada agente carga solo lo que necesita; esto importa especialmente con modelos pequeños, de ventana de contexto limitada (D.6).

Por qué no un script secuencial tradicional

Esta es la objeción más seria, porque buena parte del flujo es automatizable con código convencional, y de hecho lo está: las seis herramientas son Python determinista. La diferencia está en lo que ocurre entre una herramienta y la siguiente.

Decisión del flujo	Script tradicional	Capa MAS
Qué métrica investigar al ver una anomalía	Lista fija de métricas en orden fijo.	Elige la siguiente consulta según lo que acaba de observar.
Cómo atribuir una causa	Umbral codificado: si X > k entonces causa Y.	Cruza telemetría, topología y bitácora para distinguir configuración de demanda.
Qué parámetro ajustar y cuánto	Tabla de reglas mantenida a mano.	Deriva el ajuste del diagnóstico y lo justifica.
Comunicar el resultado	Plantilla rellenada.	Explica qué cambió, por qué y a costa de qué.

Esta ventaja es **cualitativa y condicional a la calidad del modelo**: las corridas con un LLM de 7B muestran que el Ingeniero a veces ignora los límites de la normativa y que el Sintetizador a veces inventa datos (D.9). Por eso la solución no confía en el LLM para nada verificable.

Alternativa que debe reconocerse

Para el ajuste fino de pesos y ciclos, la herramienta estándar es la **optimización paramétrica sobre el simulador** (búsqueda en rejilla, optimización bayesiana, estrategias evolutivas). Es numéricamente superior a un LLM para esa tarea. La capa MAS no la reemplaza: aporta la interpretación del diagnóstico, la auditoría frente a texto normativo y la explicación para quien decide. Cerrar el lazo con simulación (una herramienta `simular_configuracion` sobre SUMO) está en el trabajo siguiente; sin ella, las propuestas de esta fase **no se evalúan en desempeño**, solo en cumplimiento de reglas.

A.2 Ventajas de la especialización por roles

La separación en roles acotados tiene tres efectos, que se presentan junto con la evidencia disponible y sus límites.

Reducción del sesgo

El sesgo más probable es el **de confirmación sobre el propio trabajo**: quien propone un cambio tiende a evaluarlo favorablemente. El diseño lo mitiga separando proponer de auditar. El Auditor no tiene herramientas que calculen desempeño, así que no puede ponderar una mejora de métricas contra una violación normativa. Su veredicto lo emite un validador Python, no su opinión. Un segundo sesgo, el de anclaje en el promedio, se ataca en el *backstory* del Analista, que trabaja por intersección y escenario: un promedio del circuito ocultaría I3, donde está el problema sembrado en los datos.

Limitación del espacio de estados

En un agente con seis herramientas el espacio de acciones por paso es de seis opciones más la respuesta final. Repartidas, cada agente enfrenta entre cero y tres. Además cada rol restringe el espacio semántico. La tabla distingue lo que es **imposible por construcción** (no hay herramienta) de lo que **solo está pedido en el prompt**.

Agente	Herramientas	Imposible por construcción	Solo pedido en el prompt
Analista de Telemetría Vial	3	Publicar o escribir configuración.	No proponer cambios.
Ingeniero de Planes	3	Validar o publicar.	No declarar que cumple la normativa.
Auditor Normativo	3	Calcular métricas de desempeño.	No proponer planes ni opinar sobre cumplimiento.
Sintetizador de Movilidad	0	Consultar datos nuevos.	No inventar cifras: **no está garantizado**; ver verificación en D.4 y D.9.

Que el Sintetizador no tenga herramientas le quita el acceso a los datos, pero **no le impide inventar**. En las corridas de prueba lo hizo (costos monetarios, tiempos de implementación, nombres de archivo). Por eso el sistema incluye una verificación determinista posterior al Crew (`verificacion.py`).

Precisión en la ejecución

El *role-playing* convierte una instrucción genérica en un criterio operativo: «sé riguroso» no cambia el comportamiento de un modelo; «investigaste un atropellamiento causado por recortar tres segundos de verde peatonal para mejorar un indicador de flujo» fija qué compensaciones son inaceptables. Los *backstories* de este Crew codifican una regla de decisión que el agente aplica cuando el caso no está cubierto por su instrucción explícita. Es una hipótesis de diseño; el prototipo no mide su efecto aislado.

Costo. La especialización multiplica las llamadas al modelo (seis tareas en lugar de una, más reintentos): las corridas con un LLM local tardaron entre 3 y 35 minutos. Es un intercambio deliberado: en una capa que corre una vez por jornada, la verificabilidad vale más que la latencia. Habría sido inaceptable en la capa de tiempo real.

A.3 Mecanismo de orquestación

Elección: Process.sequential con lazo de corrección explícito y regla determinista de publicación

```
crew = Crew(
    agents=[analista, ingeniero, auditor, sintetizador],
    tasks=[t_diagnostico, t_propuesta_v1, t_auditoria_v1,
```

```
            t_propuesta_v2, t_auditoria_v2, t_sintesis],
    process=Process.sequential,
    verbose=True,
)
```

La naturaleza del problema impone una cadena de dependencias estricta: no se puede proponer sin diagnóstico, ni auditar sin propuesta, ni comunicar sin dictamen. El orden no es una incógnita del problema, es parte de su definición.

Opción	Qué implicaría aquí	Decisión
`Process.hierarchical`	Un agente gerente decide en cada paso a quién asignar la tarea siguiente.	**Descartada.** Resuelve un problema que aquí no existe (el orden es conocido y fijo) y agrega no determinismo, llamadas extra al modelo y un punto de fallo en una cadena que debe ser auditable paso a paso.
Delegación directa pura	Un agente raíz delega libremente y el flujo emerge de las delegaciones.	**Descartada** como mecanismo principal: hace irreproducible la traza. Se conserva como capacidad opcional del Ingeniero (ver Bloque B).
`Process.sequential`	Las seis tareas se ejecutan en orden, pasando contexto explícito.	**Elegida.** El orden de ejecución coincide con el orden lógico del problema y la traza es reproducible y auditable.

El lazo de retroalimentación sin perder el determinismo

Un proceso secuencial puro no permitiría que el Auditor devolviera trabajo al Ingeniero, y ese lazo es indispensable: un dictamen sin consecuencia no es una auditoría. La solución es desdoblar la revisión en dos pasadas y entregar el dictamen de la primera como contexto de la re-propuesta, **junto con la propuesta rechazada** (sin ella, el Ingeniero no podría corregirla ni devolverla intacta):

```
t_propuesta_v2 = Task(
    description="Si el dictamen recibido autorizo la publicacion, devuelve la misma "
                "propuesta sin cambios. Si la rechazo, corrige la propuesta atacando "
                "la causa raiz que senala el Auditor, no solo el sintoma. ...",
    agent=ingeniero,
    context=[t_diagnostico, t_propuesta_v1, t_auditoria_v1],   # <- aqui entra el veto
    output_pydantic=PropuestaConfig,
    output_file=str(DIR_SALIDAS / "propuesta_v2.json"),
)
```

El lazo está **acotado** (el número de pasadas es fijo) y es **auditable**: la propuesta rechazada, el motivo y la corregida quedan en la traza y en `salidas/`.

Regla determinista de publicación. Una corrida de prueba mostró que depender de que el LLM copie un veredicto es frágil: la propuesta v1 era válida, pero el Ingeniero la «corrigió» en v2 y la volvió inválida. Por eso el publicador acepta `archivo='ultima_valida'`: publica la propuesta v2 si pasa el validador y, si no, la v1 si esa pasa; si ninguna pasa, no publica. La decisión es de código, no del modelo.

Por qué CrewAI y no otro framework

La decisión se tomó sobre tres criterios. El primero es que el modelo conceptual de CrewAI (agente con rol, meta y trasfondo; tarea con salida esperada; equipo con proceso) corresponde de manera directa con la especificación PEAS de las fases anteriores, lo que mantiene la trazabilidad entre fases. El segundo es el sistema de herramientas: `BaseTool` con esquema `pydantic` permite que toda la lógica verificable viva en Python y que el LLM solo decida cuál invocar y con qué argumentos. El tercero es que el campo `context` de `Task` hace explícito el flujo de información entre etapas. Además, `output_pydantic` permite exigir una salida con esquema.

Frameworks de grafos de estado como LangGraph, y los *Flows* del propio CrewAI, permitirían expresar el lazo de corrección como un ciclo real en lugar de un desdoblamiento. Se descartó esa vía por el primer criterio y porque el lazo de este problema tiene exactamente dos pasadas; sería la vía natural si el número de iteraciones fuera variable.

Bloque B. Especificación técnica de los agentes especializados

En palabras simples

Cada agente es como un empleado con un puesto: tiene un **rol** (qué es), una **meta** (qué busca lograr), un **trasfondo** (su experiencia, que guía cómo razona), unas **herramientas** (lo único que puede usar para consultar datos o hacer cálculos) y un permiso de **delegación** (si puede pedir ayuda a un compañero). Aquí se describen los cuatro agentes del equipo con esos cinco datos.

El Crew está formado por cuatro agentes. Se documentan los cinco campos solicitados para cada uno, más el criterio de diseño que lo distingue. Los textos de `role`, `goal` y `backstory` del código están escritos sin acentos (ASCII); aquí se reproducen con ortografía normal.

Decisión transversal: ninguna herramienta delega juicio al modelo. Las seis son Python determinista. El LLM decide cuál invocar y con qué argumentos e interpreta el resultado, pero no calcula una métrica ni juzga el cumplimiento de una regla. Es el mismo principio que rige al Supervisor de la Fase 1: lo verificable no se delega al razonamiento.

B.1 Analista de Telemetría Vial

`role`	**Analista de Telemetría Vial**
`goal`	Determinar, con evidencia estadística y no con impresiones, en qué intersección y bajo qué escenario el desempeño de SIGMA-Vial se aparta de sus metas, y señalar si la causa está en la configuración o en la demanda.
`backstory`	Ingeniera de tránsito con doce años calibrando modelos microscópicos para municipios. Aprendió a desconfiar de los promedios: en su primer proyecto reportó una mejora del 18 % que resultó ser el efecto de dos réplicas atípicas, y desde entonces no afirma nada que no sostenga una prueba estadística. Trabaja por intersección y por escenario, nunca con el agregado del circuito, porque sabe que un promedio global esconde justo la intersección que falla.
`tools`	• `lector_telemetria_sumo`: media y desviación por intersección y controlador sobre las 10 réplicas (por omisión, 4 métricas principales). • `comparador_lineas_base`: variación porcentual frente a plan fijo y máxima presión, con prueba de Wilcoxon pareada. • `lector_contexto_operativo`: configuración vigente, topología, calendario, normativa y bitácora.
`allow_delegation`	`False`. Produce evidencia, no dirige a nadie; permitirle delegar le daría la posibilidad de pedir una solución y contaminaría el diagnóstico.
Criterio de diseño	El *backstory* ataca el sesgo de anclaje en el promedio. `max_iter=12`.

B.2 Ingeniero de Planes de Señalización

role	**Ingeniero de Planes de Señalización**
goal	Convertir el diagnóstico en una propuesta concreta y justificada de nuevos pesos de la función de utilidad y nuevos planes de semáforo, maximizando el desempeño sin inventar valores que la normativa no permita.
backstory	Formado en la escuela clásica de ingeniería de tránsito: calcula Webster a mano antes de confiar en cualquier optimizador. Pasó cinco años operando un centro de control urbano, donde aprendió que un parámetro que mejora una intersección suele empeorar la siguiente, y por eso razona siempre sobre el circuito completo. Cuando el diagnóstico que recibe no le alcanza para decidir, lo pide; no rellena el hueco con supuestos.
tools	• `calculadora_webster`: plan base por Webster (1958) **restringido por la normativa**: ciclo común y verdes que ya cumplen R0-R6. • `lector_contexto_operativo`: topología y calendario académico. • `lector_telemetria_sumo`: verificación puntual de un dato del diagnóstico.
allow_delegation	Configurable con la variable `SIGMA_DELEGACION`; **`False` por omisión**. Es el único agente al que se le puede habilitar: con `True` puede pedir al Analista un desglose adicional cuando el diagnóstico no basta para fijar un parámetro (condición: pedir en lugar de suponer). Se apagó por evidencia empírica: con `llama3.1:8b` la delegación hizo que Ingeniero y Auditor se enviaran preguntas sin aportar y que una tarea terminara con el contenido de otra (D.9).
Criterio de diseño	Es el único rol con una decisión genuinamente abierta (no existe una respuesta única a «cuánto subir el peso peatonal»). Para lo que es cálculo, la herramienta le da el punto de partida factible.

B.3 Auditor Normativo y de Seguridad Vial

role	**Auditor Normativo y de Seguridad Vial**
goal	Impedir que se publique cualquier configuración que viole una regla dura o la normativa de cruce peatonal, sin importar cuánto mejore las métricas de desempeño.
backstory	Viene de auditoría de seguridad vial, no de optimización de tráfico. Investigó un atropellamiento en un cruce cuyo tiempo de verde peatonal se había recortado tres segundos para mejorar un indicador de flujo, y esa investigación define cómo trabaja: el desempeño nunca justifica una excepción a una regla dura. No emite juicios propios sobre si un plan cumple; ejecuta el validador y reporta su veredicto literal, porque una regla dura verificada por opinión deja de ser una regla dura.
tools	• `validador_reglas_duras`: evalúa R0-R6 y emite APROBADO o RECHAZADO; recibe el nombre del archivo a validar. • `lector_contexto_operativo`: normativa vigente. • `publicador_configuracion`: escribe la configuración aprobada; **vuelve a validar internamente** y rechaza la publicación si no pasa, de modo que el veto no depende de que el agente se comporte bien.
allow_delegation	`False`. El veto no se delega: si pudiera, podría pedir a otro agente una interpretación más favorable de una regla, justo lo que el rol existe para impedir.
Criterio de diseño	Equivalente, en la capa MAS, del Supervisor de seguridad: quien verifica la seguridad no es quien propone la mejora. Doble barrera: el agente valida y el publicador revalida.

B.4 Sintetizador de Movilidad

role	**Sintetizador de Movilidad**
goal	Entregar a la coordinación operativa del campus un reporte que explique qué cambió, por qué, con qué evidencia y a costa de qué, en lenguaje comprensible para quien no es ingeniero de tránsito.
backstory	Comunicador técnico en proyectos de movilidad urbana. Su criterio es que una decisión que no se puede explicar no se puede sostener en una reunión, y que ocultar el costo de un cambio destruye la confianza más rápido que el cambio mismo. Por eso siempre escribe qué se sacrificó para obtener la mejora.
tools	Ninguna, deliberadamente: no accede a los datos. Esto **no** garantiza que no invente cifras; el reporte pasa por una verificación determinista posterior (D.4).
allow_delegation	`False`. Su insumo es el producto de las tareas previas; delegar solo reabriría decisiones ya tomadas y auditadas.
Criterio de diseño	El único agente cuya meta incluye declarar el costo del cambio. `max_iter=8`.

B.5 Catálogo de herramientas

Herramienta	Qué hace (Python determinista)	Agentes
`lector_telemetria_sumo`	Media y desviación de métricas por intersección y controlador sobre 10 réplicas.	Analista, Ingeniero
`comparador_lineas_base`	Variación porcentual de SIGMA-Vial frente a las dos líneas base y valor p de Wilcoxon pareada (`scipy.stats`).	Analista
`lector_contexto_operativo`	Lee configuración, normativa, calendario, topología o bitácora.	Analista, Ingeniero, Auditor
`calculadora_webster`	Plan base de Webster restringido por la normativa: menor ciclo común factible y verdes que cumplen R0-R6 y capacidad (grado de saturación 0.9).	Ingeniero
`validador_reglas_duras`	Evalúa R0 a R6 y emite veredicto. Única fuente autorizada para declarar cumplimiento. Lee la propuesta por nombre de archivo.	Auditor
`publicador_configuracion`	Revalida y escribe `configuracion_aprobada.json`. Con `archivo='ultima_valida'` publica la propuesta más reciente que cumpla.	Auditor

Reglas duras que evalúa el validador

Provienen de la ficha de las Fases 1-2. **R0 y R6 son nuevas respecto a la primera versión de este prototipo:** R6 (espera máxima por carril) estaba en la ficha pero no se evaluaba, y R0 se agregó al detectar, en una corrida real, que una propuesta con solo 2 de las 4 intersecciones pasaba la auditoría.

Regla	Condición	Origen
R0 Esquema y completitud	Las 4 intersecciones presentes; `pesos_utilidad` con las 5 claves de la configuración vigente, entre 0 y 5; `umbral_spillback_pct` entre 0.3 y 0.95 (fracción de la capacidad del tramo); `periodo_decision_s` entre 1 y 10.	Prototipo
R1 Verde vehicular	8 s ≤ verde ≤ 60 s en cada acceso.	Fase 1-2
R2 Ciclo máximo	Ciclo ≤ 120 s.	Fase 1-2
R3 Despeje peatonal	Verde peatonal ≥ ancho del cruce / 1.0 m/s.	Fase 1-2
R4 Consistencia de fases	Suma de verdes + 3 × (ámbar 3 s + todo-rojo 2 s) ≤ ciclo.	Fase 1-2
R5 Espera peatonal máxima	Ciclo − verde peatonal ≤ 60 s (fase peatonal una vez por ciclo).	Fase 1-2
R6 Espera máxima por carril	Ciclo − verde del acceso ≤ 60 s.	Fase 1-2

Consecuencia analítica de R5 y R6 (hallazgo de diseño, no de la corrida)

Con N fases exclusivas, pérdidas L = 5·N s y espera máxima E = 60 s, cada verde debe ser al menos ciclo − E; sumando, ciclo ≤ (N·E − L)/(N − 1). Para N = 3 fases el ciclo máximo factible es **82.5 s**, muy por debajo del máximo de 120 s de la ficha. No es una contradicción lógica, pero el ciclo de 120 s solo sería alcanzable con fases peatonales concurrentes con el tráfico paralelo o servidas dos veces por ciclo. **Recomendamos revisar la especificación de las Fases 1-2** en ese punto.

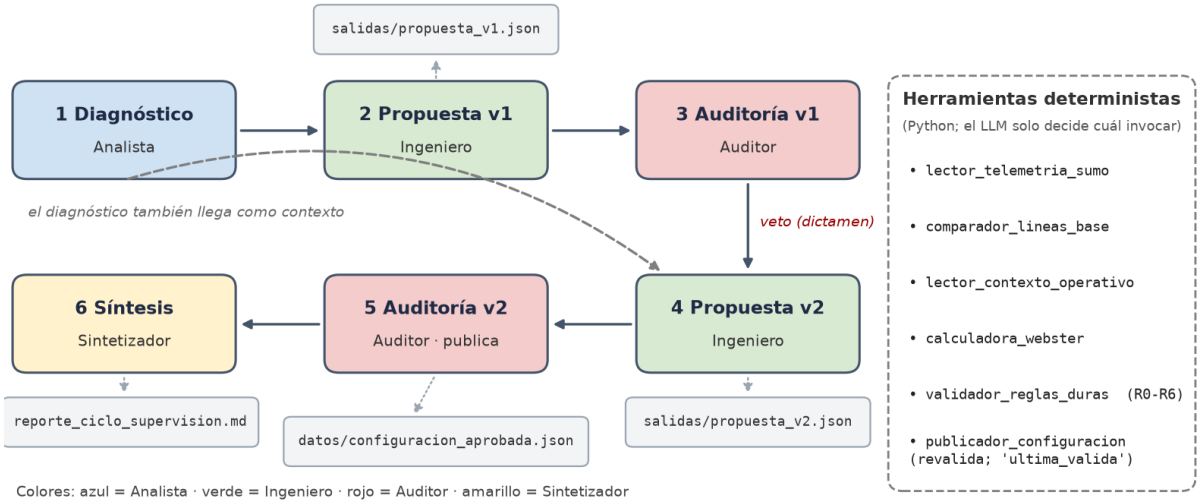
Los parámetros (ámbar 3 s, todo-rojo 2 s, 1.0 m/s, esperas de 60 s) son los de la ficha, marcados allí como «[Confirmar con normativa local]»; son parámetros de diseño del proyecto, no una cita normativa verificada.

Bloque C. Desglose de tareas especializadas y flujo de información

En palabras simples

Una **tarea** es una instrucción concreta para un agente: qué debe hacer (*description*), en qué formato debe entregar el resultado (*expected_output*), quién la hace (*agent*) y qué resultados anteriores puede leer (*context*). Las seis tareas se ejecutan en orden: cada una recibe lo que produjo la anterior.

C.1 Flujo general



El diagnóstico alimenta tanto la primera propuesta como la re-propuesta, de modo que la corrección no pierde de vista el problema original mientras atiende el veto. Las propuestas se guardan en `salidas/` y las herramientas de auditoría las leen por nombre.

C.2 Tarea 1 — Diagnóstico de desempeño

`description`	**Diagnostica el desempeño de SIGMA-Vial en el escenario {escenario}. 1) Lee la configuración vigente. 2) Lee la telemetría del escenario y compara SIGMA-Vial contra las dos líneas base en espera peatonal y en eventos de spillback. 3) Revisa la bitácora del Supervisor. Reporta solo diferencias con p < 0.05 en la prueba de Wilcoxon. Distingue si la causa probable es la configuración o la demanda.**
`expected_output`	JSON con escenario_analizado, replicas, hallazgos (por intersección: variación porcentual en espera peatonal, espera vehicular y spillback, p_wilcoxon y severidad), conclusion y prioridad_de_atencion.
`agent`	Analista de Telemetría Vial
Dependencias	Ninguna: es la raíz. Recibe `{escenario}` como variable de entrada del `kickoff`.

El umbral p < 0.05 está en la descripción y no en el código a propósito: es un criterio metodológico que el equipo puede querer ajustar sin tocar la herramienta. Nota estadística: con 10 réplicas pareadas el valor p exacto mínimo del test de Wilcoxon es 0.00195, de modo que ese valor significa «las 10 réplicas van en el mismo sentido» y no dice nada sobre la magnitud; no se aplicó corrección por comparaciones múltiples.

C.3 Tarea 2 — Propuesta de ajuste (ciclo 1)

`description`	**Con el diagnóstico recibido, propón una nueva versión de la configuración. Consulta la topología y el calendario. Para los planes de semáforo llama primero a `calculadora_webster` (sin argumentos) y úsala como base sin salir de sus límites; tu aporte son los pesos de utilidad, el umbral de spillback y el desfase de cada intersección. Incluye las cuatro intersecciones y los pesos de la configuración vigente. Cada parámetro que cambies lleva su justificación. Responde únicamente con el JSON.**
`expected_output`	JSON con version, pesos_utilidad, planes_por_interseccion (ciclo_s, verde_principal_s, verde_secundario_s, verde_peatonal_s, desfase_s por intersección), umbral_spillback_pct, periodo_decision_s y justificacion. Se fuerza con `output_pydantic=PropuestaConfig` y se guarda en `salidas/propuesta_v1.json`.
`agent`	Ingeniero de Planes de Señalización
Dependencias	`context=[t_diagnostico]`

C.4 Tarea 3 — Auditoría normativa (ciclo 1)

`description`	**Audita la propuesta contra las reglas duras. 1) Lee la normativa. 2) Valida PRIMERO la configuración vigente (`archivo='vigente'`), para dejar constancia de si lo que hoy se ejecuta cumple. 3) Valida la propuesta (`archivo='propuesta_v1'`). Reporta el veredicto literal del validador; no lo sustituyas por tu criterio ni apruebes nada por mejora de desempeño. Si rechazas, da al Ingeniero una instrucción sobre la causa raíz.**
`expected_output`	JSON con ciclo_de_auditoria, dictamen_configuracion_vigente, dictamen_propuesta, reglas_evaluadas, autorizacion_de_publicacion y, si procede, instruccion_al_ingeniero.
`agent`	Auditor Normativo y de Seguridad Vial
Dependencias	`context=[t_propuesta_v1]`

Auditar también la configuración vigente (paso 2) no es necesario para evaluar la propuesta, pero deja constancia del estado actual. La lección de diseño es que una auditoría debe revisar el estado presente, no solo el cambio propuesto.

C.5 Tarea 4 — Corrección (ciclo 2)

`description`	**Si el dictamen recibido autorizó la publicación, devuelve la misma propuesta sin cambios. Si la rechazó, corrige atacando la causa raíz que señala el Auditor, no solo el síntoma. Consulta la normativa. Documenta qué reveló el veto. Responde únicamente con el JSON.**
`expected_output`	Misma estructura de la propuesta anterior, con versión incrementada y la justificación, incluido el costo asumido. `output_pydantic` y `salidas/propuesta_v2.json`.
`agent`	Ingeniero de Planes de Señalización
Dependencias	`context=[t_diagnostico, t_propuesta_v1, t_auditoria_v1]`: recibe el problema original, la propuesta rechazada y el veto.

La condición «devuelve la misma si fue aprobada» hace que el flujo de seis tareas funcione igual cuando no hay veto, sin ramificar el grafo.

C.6 Tarea 5 — Auditoría y publicación (ciclo 2)

`description`	**Audita la propuesta corregida con `validador_reglas_duras` (`archivo='propuesta_v2'`). Después llama a `publicador_configuracion` con `archivo='ultima_valida'`: publica la v2 si cumple y, si no, la v1 si esa cumple; si ninguna cumple, no publica. Reporta el veredicto literal y la fuente publicada.**
`expected_output`	JSON con ciclo_de_auditoria, dictamen_propuesta, reglas_evaluadas, autorizacion_de_publicacion y publicacion.
`agent`	Auditor Normativo y de Seguridad Vial
Dependencias	`context=[t_propuesta_v2]`

C.7 Tarea 6 — Síntesis para la coordinación

`description`	**Redacta el reporte del ciclo para la coordinación del campus: hallazgo principal, hallazgos de seguridad, por qué se rechazó la primera propuesta, qué se cambió y a costa de qué, y trazabilidad. Reglas: solo cifras presentes en el contexto; sin tabla de parámetros (la agrega el sistema, generada por código); sin costos monetarios, horas ni mejoras cuantificadas inventadas; sin afirmar cumplimientos que el dictamen no confirme.**
`expected_output`	Reporte en Markdown con encabezados, sección de costo asumido y trazabilidad. Se escribe a `reporte_ciclo_supervision.md`.
`agent`	Sintetizador de Movilidad
Dependencias	`context=[t_diagnostico, t_propuesta_v1, t_auditoria_v1, t_propuesta_v2, t_auditoria_v2]`
Postproceso	`verificacion.py` (código, sin LLM): antepone el estado de publicación, marca cifras sin respaldo y archivos citados inexistentes, y agrega la tabla antes/después generada desde los JSON.

C.8 Resumen del flujo de información

#	Tarea	Agente	Contexto que consume	Salida a disco
1	Diagnóstico	Analista	Variable `{escenario}`	—
2	Propuesta v1	Ingeniero	Tarea 1	`salidas/propuesta_v1.json`
3	Auditoría ciclo 1	Auditor	Tarea 2	—
4	Propuesta v2	Ingeniero	Tareas 1, 2 y 3	`salidas/propuesta_v2.json`
5	Auditoría ciclo 2 y publicación	Auditor	Tarea 4	`datos/configuracion_aprobada.json`
6	Síntesis	Sintetizador	Tareas 1 a 5	`reporte_ciclo_supervision.md`

Bloque D. Implementación del prototipo en Python con CrewAI

En palabras simples

Aquí se muestra el programa real y una ejecución completa. Primero el código que crea a los agentes, las tareas y el equipo; luego las piezas de seguridad que no dependen de la inteligencia artificial (las «herramientas»); y al final lo que pasó cuando se ejecutó: qué recibió el equipo, cómo trabajó cada agente y qué reporte entregó. Se incluyen también las ejecuciones que fallaron, porque explican las decisiones de diseño.

D.1 Estructura del proyecto

```
SIGMA_Vial_Fase3_codigo/
├── sigma_crew.py             Agentes, tareas, Crew, guardrail — punto de entrada (ejecutar())
├── herramientas.py           6 herramientas BaseTool, deterministas
├── verificacion.py           Verificación determinista del reporte final (sin LLM)
├── generar_telemetria.py     Telemetría sintética (semilla fija) y datos de entrada
├── llm_stub.py               LLM de guion fijo, solo para probar el código sin modelo
├── Modelfile.qwen25          Alias de Ollama (qwen2.5:7b, contexto 16k) — modelo de la demo
├── requirements.txt          Versiones fijadas
├── SIGMA_Vial_Fase3.ipynb    Notebook ejecutable, con salidas de la corrida
├── README.md
├── datos/                    metricas_simulacion.csv (480 filas), bitacora_supervisor.csv,
│                             configuracion_vigente.json, configuracion_aprobada.json (salida),
│                             normativa.json, calendario_academico.json, topologia_circuito.json
├── salidas/                  propuesta_v1.json, propuesta_v2.json, tarea_*.txt (salida de cada tarea)
├── corrida_demostracion.log  Log completo de la corrida del notebook
└── reporte_ciclo_supervision.md   Reporte del Sintetizador + verificación
```

D.2 Instanciación de los agentes

Se muestra un agente completo y el extracto diferencial de los otros tres; el archivo íntegro está en el repositorio.

```
analista = Agent(
    role="Analista de Telemetria Vial",
    goal=(
        "Determinar, con evidencia estadistica y no con impresiones, en que "
        "interseccion y bajo que escenario el desempeno de SIGMA-Vial se aparta "
        "de sus metas, y senalar si la causa esta en la configuracion o en la "
        "demanda."
    ),
    backstory=(
        "Ingeniera de transito con doce anios calibrando modelos microscopicos "
        "para municipios. Aprendio a desconfiar de los promedios: en su primer "
        "proyecto reporto una mejora del 18% que resulto ser el efecto de dos "
        "replicas atipicas, y desde entonces no afirma nada que no sostenga una "
        "prueba estadistica. Trabaja por interseccion y por escenario, nunca con "
        "el agregado del circuito, porque sabe que un promedio global esconde "
        "justo la interseccion que falla."
    ),
    tools=[HERRAMIENTAS["telemetria"], HERRAMIENTAS["comparador"],
           HERRAMIENTAS["contexto"]],
    llm=LLM_CREW,
    allow_delegation=False,   # produce evidencia; no dirige a nadie
    verbose=True,
    max_iter=12,
)
```

```
# ... role, goal y backstory como en el Bloque B ...
ingeniero = Agent(
    role="Ingeniero de Planes de Senalizacion",
    goal=(...), backstory=(...),
    tools=[HERRAMIENTAS["webster"], HERRAMIENTAS["contexto"],
           HERRAMIENTAS["telemetria"]],
    llm=LLM_CREW,
    # Unico agente al que se le puede habilitar delegacion: pedir al Analista
    # un desglose adicional cuando el diagnostico no basta (ver arriba).
```

```
    allow_delegation=DELEGACION_INGENIERO,
    verbose=True,
    max_iter=12,
)
```

```
auditor = Agent(
    role="Auditor Normativo y de Seguridad Vial",
    goal=(...), backstory=(...),
    tools=[HERRAMIENTAS["validador"], HERRAMIENTAS["contexto"],
           HERRAMIENTAS["publicador"]],
    llm=LLM_CREW,
    allow_delegation=False,   # el veto no se delega
    verbose=True,
    max_iter=12,
)
```

```
sintetizador = Agent(
    role="Sintetizador de Movilidad",
    goal=(...), backstory=(...),
    tools=[],   # solo sintetiza el contexto que recibe; no toca los datos
    llm=LLM_CREW,
    allow_delegation=False,
    verbose=True,
    max_iter=8,
)
```

D.3 Definición de las tareas

Dos tareas representativas; las seis están en `sigma_crew.py` con su `description` y `expected_output` completos (resumidos en el Bloque C).

```
t_propuesta_v1 = Task(
    description=(
        "Con el diagnostico recibido, propon una nueva version de la "
        "configuracion. Ajusta los pesos de la funcion de utilidad y los planes "
        "de las intersecciones senaladas como prioritarias. Consulta la "
        "topologia para considerar la longitud de los tramos y el calendario "
        "academico para los pulsos de demanda.\n"
        "Llama PRIMERO a calculadora_webster (sin argumentos). Devuelve "
        "'propuesta_base': una configuracion completa que ya cumple las "
        "reglas duras (cuatro intersecciones, ciclo comun, verdes y desfases). "
        "Copiala tal cual y modifica SOLO lo que el diagnostico justifique: "
        "los pesos de utilidad (cada peso entre 0 y 5; si el diagnostico "
        "muestra que la espera peatonal EMPEORA en alguna interseccion, sube "
        "w_espera_peat, y si el spillback EMPEORA sube "
        "w_ocupacion_aguas_abajo; si nada empeora deja los pesos como estan; "
        "justifica cada cambio con la cifra del diagnostico) y, si hace falta, "
        "umbral_spillback_pct (fraccion de la capacidad del tramo, entre 0.3 y "
        "0.95). NO cambies planes_por_interseccion ni periodo_decision_s: "
        "cualquier cambio ahi las invalida. Pon version 'v1.1' y llena "
        "'justificacion' con una entrada por cada parametro que cambies. "
        "Cada parametro que cambies debe llevar su "
        "justificacion. No propongas valores fuera de la normativa. "
        "Responde unicamente con el JSON, sin texto adicional."
    ),
    expected_output=(
        "JSON con version, pesos_utilidad, planes_por_interseccion (ciclo_s, "
        "verde_principal_s, verde_secundario_s, verde_peatonal_s, desfase_s "
        "por interseccion), umbral_spillback_pct, periodo_decision_s y un "
        "objeto justificacion con una entrada por parametro modificado."
    ),
    agent=ingeniero,
    context=[t_diagnostico],       # depende de la salida del diagnostico
    output_pydantic=PropuestaConfig,
    guardrail=(None if os.getenv("SIGMA_LLM", "stub").lower() == "stub"
               else _guardrail_propuesta("v1")),
    output_file=str(DIR_SALIDAS / "propuesta_v1.json"),
)
```

```
t_auditoria_v2 = Task(
    description=(
        "Audita la propuesta corregida. Valida con validador_reglas_duras "
        "(archivo='propuesta_v2'). Despues llama a publicador_configuracion "
        "con archivo='ultima_valida': publica la propuesta v2 si cumple las "
```

```
        "reglas duras y, si no, la v1 si esa si las cumple; si ninguna las "
        "cumple, no publica nada. Reporta el veredicto literal del validador "
        "y el resultado de la publicacion, incluida la fuente publicada."
    ),
    expected_output=(
        "JSON con ciclo_de_auditoria, dictamen_propuesta (veredicto, "
        "violaciones, advertencias), reglas_evaluadas, "
        "autorizacion_de_publicacion y publicacion (resultado del publicador)."
    ),
    agent=auditor,
    context=[t_propuesta_v2],
)
```

D.4 Herramientas deterministas y guardrails

El validador de reglas duras

Es el «inspector» del sistema: una función de Python que revisa una propuesta contra una lista de reglas (R0 a R6, descritas en B.5) y responde APROBADO o RECHAZADO con la lista de violaciones. Es la herramienta sobre la que descansa la seguridad: el agente no puede aprobar nada que esta función rechace. Se muestran las reglas R3 a R6; el resto está en `herramientas.py`.

```
        for inter, p in planes.items():          # una pasada por intersección
            ...
            # R3 — tiempo de despeje peatonal
            ancho = n["anchos_cruce_m"].get(inter)
            if ancho is None:
                advertencias.append(f"{inter}: sin ancho de cruce registrado")
            else:
                minimo = ancho / n["velocidad_cruce_peatonal_m_s"]
                if vpeat < minimo:
                    violaciones.append(
                        f"{inter}: verde peatonal {vpeat}s < despeje minimo "
                        f"{minimo:.1f}s para un cruce de {ancho}m a "
                        f"{n['velocidad_cruce_peatonal_m_s']} m/s (R3)")


            # R4 — consistencia del ciclo (fases + perdidas)
            perdidas = ((n["ambar_s"] + n["todo_rojo_s"])
                        * n.get("fases_por_ciclo", 3))
            suma = vp + vs + vpeat + perdidas
            if suma > ciclo:
                violaciones.append(
                    f"{inter}: las fases suman {suma}s (incluye {perdidas}s de "
                    f"ambar y todo-rojo) y exceden el ciclo de {ciclo}s (R4)")


            # R5 — espera maxima implicita por cruce peatonal
            espera_peat_max = ciclo - vpeat
            if espera_peat_max > n["espera_max_cruce_peatonal_s"]:
                violaciones.append(
                    f"{inter}: espera peatonal maxima implicita "
                    f"{espera_peat_max}s > limite "
                    f"{n['espera_max_cruce_peatonal_s']}s (R5)")


            # R6 — espera maxima por carril. La Fase 1-2 fija 60 s por carril
            # igual que por cruce peatonal; un acceso espera, como maximo, el
            # ciclo menos su propio verde.
            for etiqueta, v in (("principal", vp), ("secundario", vs)):
                espera_carril = ciclo - v
                if espera_carril > n["espera_max_carril_s"]:
                    violaciones.append(
                        f"{inter}: espera maxima del acceso {etiqueta} "
                        f"{espera_carril}s > limite "
                        f"{n['espera_max_carril_s']}s (R6)")
```

Publicación con segunda barrera y regla determinista

```
        if archivo == "ultima_valida":
            # Regla determinista: se publica la propuesta mas reciente que
            # pase el validador. Evita que una "correccion" del Ingeniero que
            # empeora una propuesta ya valida deje el ciclo sin publicar.
            archivo = None
            for candidato in ("propuesta_v2", "propuesta_v1"):
                if json.loads(ValidadorReglasDuras()._run(archivo=candidato)
                              )["veredicto"] == "APROBADO":
```

```
                    archivo = candidato
                    break
            if archivo is None:
                return _json({"publicado": False,
                              "motivo": "ninguna propuesta pasa las reglas "
                                        "duras"})
                # Segunda barrera: se revalida aqui, de modo que el veto no depende de
        # que el agente Auditor se comporte bien.
        dictamen = json.loads(ValidadorReglasDuras()._run(
            archivo=archivo, propuesta_json=configuracion_json))
        if dictamen["veredicto"] != "APROBADO":
            return _json({"publicado": False,
                          "motivo": "la configuracion no pasa las reglas duras",
                          "violaciones": dictamen["violaciones"]})
        texto = _resolver_propuesta(archivo, configuracion_json)
        destino = DIR_DATOS / "configuracion_aprobada.json"
        with open(destino, "w", encoding="utf-8") as f:
            f.write(texto)
        ...
        return _json({"publicado": True, "archivo": ..., "fuente": archivo})
```

Guardrail de las propuestas

Un *guardrail* («barandal») es una comprobación automática que se aplica a la respuesta de un agente antes de aceptarla. Los planes de semáforo (ciclo, verdes, desfases) no los inventa el modelo: los calcula `calculadora_webster` y ya cumplen las reglas. Al Ingeniero le toca ajustar los pesos y justificarlos. Si el modelo cambia los planes, el guardrail le devuelve un error con los valores exactos que debe usar (dos intentos) y, como último recurso, el código los corrige y lo anota. Con el stub no se aplica, porque su propuesta v1.1 es inválida a propósito para mostrar el veto.

```
def _guardrail_propuesta(nombre: str):
    def guardrail(salida) -> tuple[bool, Any]:
        base = calcular_planes_factibles()["propuesta_base"]
        try:
            prop = json.loads(_extraer_json(str(salida.raw)))
        except ValueError:
            return False, "Responde unicamente con el JSON de la propuesta."


        problemas = []
        if prop.get("planes_por_interseccion") != base["planes_por_interseccion"]:
            problemas.append("planes_por_interseccion")
        if prop.get("periodo_decision_s") != base["periodo_decision_s"]:
            problemas.append("periodo_decision_s")
        umbral = prop.get("umbral_spillback_pct")
        if not isinstance(umbral, (int, float)) or not _UMBRAL_MIN <= umbral <= _UMBRAL_MAX:
            problemas.append("umbral_spillback_pct")
        pesos = prop.get("pesos_utilidad", {})
        if any(not isinstance(v, (int, float)) or not 0 <= v <= _PESO_MAX
               for v in pesos.values()):
            problemas.append("pesos_utilidad")        if not problemas:
            return True, str(salida.raw)      # todo bien: se acepta
        ...          # 2 intentos con mensaje de error; al tercero, el código fija los valores
```

Verificación del reporte final

Que el Sintetizador no tenga herramientas no le impide inventar datos. Por eso, después de que el equipo termina, el programa `verificacion.py` (que no usa IA) revisa el reporte: antepone si se publicó o no una configuración, lista las cifras que no aparecen en ninguna salida ni dato, señala los archivos citados que no existen y agrega la tabla antes/después construida directamente de los archivos JSON.

D.5 Ensamblaje y ejecución del Crew

```
crew = Crew(
    agents=[analista, ingeniero, auditor, sintetizador],
    tasks=[t_diagnostico, t_propuesta_v1, t_auditoria_v1,
           t_propuesta_v2, t_auditoria_v2, t_sintesis],
    process=Process.sequential,
    verbose=True,
    tracing=False,   # evita que CrewAI pregunte si comparte la traza (20 s)
)
```

```
def ejecutar(escenario: str = "pico_matutino"):
    """Corrida completa: limpia salidas previas, ejecuta el Crew y verifica el
    reporte del Sintetizador con codigo determinista (ver verificacion.py).
    Es el punto de entrada que usan tanto main() como el notebook.

    Jupyter y Colab ya tienen un bucle de eventos corriendo, y CrewAI 1.15
    rechaza crew.kickoff() sincrono dentro de uno. En ese caso se ejecuta en
    un hilo aparte, que no tiene bucle."""
    try:
        asyncio.get_running_loop()
    except RuntimeError:
        return _ejecutar_sync(escenario)
    with concurrent.futures.ThreadPoolExecutor(max_workers=1) as pool:
        return pool.submit(_ejecutar_sync, escenario).result()


def _ejecutar_sync(escenario: str):
    # Un modelo local pequeno a veces devuelve una respuesta vacia o una
    # llamada a herramienta mal formada (Ollama responde 500) y CrewAI aborta
    # toda la corrida. Se reintenta la corrida completa hasta 3 veces.
    for intento in (1, 2, 3):
        limpiar_salidas()
        try:
            resultado = crew.kickoff(inputs={"escenario": escenario})
            break
        except Exception as error:      # noqa: BLE001
            print(f"[reintento {intento}/3] La corrida fallo: "
                  f"{type(error).__name__}: {str(error)[:200]}")
            if intento == 3:
                raise
    # Se excluye la ultima tarea (el reporte): sus cifras son lo que se verifica.
    fuentes = [str(t.output.raw) for t in crew.tasks[:-1] if t.output]
    # Trazabilidad: la salida textual de cada tarea queda en salidas/.
    nombres = ["diagnostico", "propuesta_v1", "auditoria_v1",
               "propuesta_v2", "auditoria_v2", "sintesis"]
    for i, (t, nombre) in enumerate(zip(crew.tasks, nombres), 1):
        if t.output:
            (DIR_SALIDAS / f"tarea_{i}_{nombre}.txt").write_text(
                str(t.output.raw), encoding="utf-8")
    print(verificar_reporte(DIR_REPORTE, fuentes))
    return resultado
```

D.6 Selección del modelo de lenguaje

El modelo se elige con la variable de entorno `SIGMA_LLM`. El modo por omisión es el stub (sin clave de API); la corrida de demostración usa **Ollama con `qwen2.5:7b`** (Qwen Team, 2024).

```
def construir_llm() -> LLM:
    proveedor = os.getenv("SIGMA_LLM", "stub").lower()

    if proveedor == "stub":
        # Servidor local determinista. El Crew, los agentes, las tareas y las
        # llamadas a herramientas son reales; solo el razonamiento esta guiado
        # por un guion fijo, para que la corrida sea reproducible y sin costo.
        import llm_stub
        llm_stub.iniciar_en_hilo()
        return LLM(model="openai/sigma-stub",
                   base_url=f"http://127.0.0.1:{llm_stub.PUERTO}/v1",
                   api_key="stub-sin-clave",
                   temperature=0.0)

    if proveedor == "ollama":
        # sigma-qwen25 = qwen2.5:7b con num_ctx ampliado (ver Modelfile.qwen25).
        # El contexto por omision de Ollama es corto y truncaria los prompts
        # del Crew sin avisar.
        modelo = os.getenv("SIGMA_MODELO", "sigma-qwen25")
        return LLM(model=f"ollama/{modelo}",
                   base_url=os.getenv("OLLAMA_HOST", "http://localhost:11434"),
                   temperature=0.2,
                   # Un modelo de 7B puede entrar en un bucle de generacion
                   # que no termina: en una corrida cada llamada agoto los
```

```
                # 10 min del servidor y el Crew quedo colgado media hora.
                # El tope de tokens y el timeout hacen que falle rapido y
                # CrewAI reintente.
                max_tokens=3000,
                timeout=240)
```

Parámetro	Valor
Modelo	`qwen2.5:7b` (7.6 B parámetros), cuantización Q4_K_M, soporte de *tool calling*; alias `sigma-qwen25` creado con `Modelfile.qwen25`
Contexto	16 384 tokens (`num_ctx`). El contexto por omisión de Ollama es corto y trunca en silencio los prompts de CrewAI.
Generación	`temperature=0.2`, `max_tokens=3000`, `timeout=240 s`. Sin estos topes, una generación en bucle agotó los 10 min del servidor.
Equipo	Windows 11, GPU NVIDIA RTX 5050 Laptop de 8 GB, 15.7 GB de RAM. El modelo cabe completo en GPU.
Elección	Se probó antes `llama3.1:8b` y se descartó (D.9). `qwen2.5:14b` no cabe en 8 GB de VRAM.

Declaración de alcance

El modo **stub** (`llm_stub.py`) sustituye solo el motor de razonamiento por un guion fijo: el Crew, las tareas, el paso de contexto y las herramientas son reales y los veredictos los emite Python, pero **no demuestra razonamiento autónomo**. Es una herramienta de prueba, no la corrida de demostración. La demostración de este documento corre con un LLM real.

D.7 Corrida de demostración

Entradas

Entrada	Contenido
Variable de kickoff	`{"escenario": "pico_matutino"}`
`metricas_simulacion.csv`	480 filas: 4 escenarios × 4 intersecciones × 3 controladores × 10 réplicas, con 10 métricas por fila.
`configuracion_vigente.json`	Versión v1.0-fase2: pesos de utilidad y planes de las cuatro intersecciones.
`normativa.json`	Reglas duras, anchos de cruce y velocidad peatonal de 1.0 m/s; límites de esquema (R0).
`bitacora_supervisor.csv`	32 incidentes: vetos, fallas de sensor, prioridades de emergencia y spillback.
`topologia_circuito.json`	Cuatro intersecciones y cuatro tramos con su capacidad en vehículos.
Modelo	`qwen2.5:7b` vía Ollama, ejecutado desde el notebook `SIGMA_Vial_Fase3.ipynb`.

Origen de los datos

La telemetría es **sintética**, generada con semilla fija por `generar_telemetria.py`, porque el modelo del circuito en SUMO aún no está construido. Los nombres y volúmenes de las intersecciones (I1 a I4) son marcadores de posición. **El desajuste de I3 (espera peatonal ×2.35 en SIGMA-Vial) y la configuración vigente son datos de prueba sembrados por el equipo**, no hallazgos sobre el circuito real ni sobre el trabajo de las Fases 1-2. Lo que la corrida demuestra es el funcionamiento de la arquitectura.

Log de interacción (extracto)

Traza condensada, generada a partir del log real. El log completo, con argumentos y salidas de cada herramienta, está en `corrida_demostracion.log` (1 938 líneas).

```
[AGENTE] Analista de Telemetria Vial
   -> Tool call: lector_contexto_operativo  {'recurso': 'configuracion'}
   -> Tool call: lector_telemetria_sumo  {'escenario': 'pico_matutino'}
   -> Tool call: comparador_lineas_base  {'escenario': 'pico_matutino', 'metrica': 'espera_peat_media_s'}
   -> Tool call: comparador_lineas_base  {'escenario': 'pico_matutino', 'metrica': 'spillback_eventos_h'}
   -> Tool call: lector_contexto_operativo  {'recurso': 'bitacora'}
[AGENTE] Ingeniero de Planes de Senalizacion
   -> Tool call: calculadora_webster
[AGENTE] Auditor Normativo y de Seguridad Vial
   -> Tool call: lector_contexto_operativo  {'recurso': 'normativa'}
   -> Tool call: validador_reglas_duras  {'archivo': 'vigente'}
   -> Tool call: validador_reglas_duras  {'archivo': 'propuesta_v1'}
[AGENTE] Ingeniero de Planes de Senalizacion
[AGENTE] Auditor Normativo y de Seguridad Vial
   -> Tool call: validador_reglas_duras  {'archivo': 'propuesta_v2'}
   -> Tool call: publicador_configuracion  {'archivo': 'ultima_valida'}
[AGENTE] Sintetizador de Movilidad
```

Resultados intermedios

Tarea 1 — Diagnóstico. Resumen de lo que consultó el Analista, calculado por la herramienta `comparador_lineas_base` sobre 10 réplicas (variación de SIGMA-Vial frente a plan fijo de Webster, y p de Wilcoxon):

Interseccion	Espera peatonal	Espera vehicular	Spillback (eventos/h)
I1	-37.4 % (mejora; p=0.00195)	-51.8 % (mejora; p=0.00195)	-81.4 % (mejora; p=0.00391)
I2	-26.9 % (mejora; p=0.00977)	-44.7 % (mejora; p=0.00195)	+105.6 % (sin diferencia significativa; p=0.10547)
I3	+64.3 % (empeora; p=0.00195)	-43.0 % (mejora; p=0.00195)	+96.8 % (sin diferencia significativa; p=0.57812)
I4	-29.3 % (mejora; p=0.00195)	-52.2 % (mejora; p=0.00195)	-76.8 % (mejora; p=0.00586)

Texto del diagnóstico del Analista (extracto de su respuesta final):

```
escenario_analizado: pico_matutino   replicas: 10
  I1: severidad = baja
  I2: severidad = alta
  I3: severidad = alta
  I4: severidad = baja
conclusion: Intersección I3 y I2 presentan problemas significativos en el pico matutino, con mayor demanda de
    peatones y eventos de spillback. Se recomienda revisar la configuración o ajustar la demanda en estas
    intersecciones.
prioridad_de_atencion: ['I3', 'I2']
```

Tarea 3 — Auditoría del ciclo 1. El Auditor validó primero la configuración vigente y después la propuesta v1. Veredictos literales del validador (R0 a R6):

```
{
  "ciclo_de_auditoria": "Auditoría de Propuesta vs Normativa",
  "dictamen_configuracion_vigente": {
    "veredicto": "RECHAZADO",
    "violaciones": [
      "I1: las fases suman 103s (incluye 15s de ambar y todo-rojo) y exceden el ciclo de 95s (R4)",
      "I1: espera peatonal maxima implicita 79s > limite 60s (R5)",
      "I1: espera maxima del acceso secundario 67s > limite 60s (R6)",
      "I2: las fases suman 93s (incluye 15s de ambar y todo-rojo) y exceden el ciclo de 85s (R4)",
      "I2: espera peatonal maxima implicita 71s > limite 60s (R5)",
      "I2: espera maxima del acceso secundario 61s > limite 60s (R6)",
      "I3: verde peatonal 11s < despeje minimo 14.0s para un cruce de 14.0m a 1.0 m/s (R3)",
      "I3: las fases suman 94s (incluye 15s de ambar y todo-rojo) y exceden el ciclo de 85s (R4)",
      "I3: espera peatonal maxima implicita 74s > limite 60s (R5)",
      "I3: espera maxima del acceso secundario 63s > limite 60s (R6)",
      "I4: las fases suman 103s (incluye 15s de ambar y todo-rojo) y exceden el ciclo de 95s (R4)",
```

```
      "I4: espera peatonal maxima implicita 83s > limite 60s (R5)",
      "I4: espera maxima del acceso secundario 69s > limite 60s (R6)"
    ],
    "observacion": "La configuración vigente viola varias reglas duras, incluyendo exceso de tiempo de ciclo, espera
        peatonal y espera máxima por carril."
  },
  "dictamen_propuesta": {
    "veredicto": "APROBADO",
    "violaciones": [],
    "observacion": "La propuesta v1 cumple con todas las reglas duras y la normativa de cruce peatonal."
  },
  "reglas_evaluadas": [
    "R0 esquema y completitud",
    "R1 verde vehicular",
    "R2 ciclo maximo",
    "R3 despeje peatonal",
    "R4 consistencia de fases",
    "R5 espera peatonal maxima",
    "R6 espera maxima por carril"
  ],
  "autorizacion_de_publicacion": true,
  "instruccion_al_ingeniero": "La propuesta v1 cumple con todas las reglas duras y la normativa de cruce peatonal. Por
        favor, proceda con la publicación de esta configuración."
}
```


Tarea 2/4 — Propuesta del Ingeniero (`salidas/propuesta_v1.json` y `propuesta_v2.json`). Parámetros tras la tarea 4:

Parametro	vigente	propuesta v1	propuesta v2	publicada
w_veh_atendidos	1.0	1.0	1.0	1.0
w_peat_atendidos	1.0	1.0	1.0	1.0
w_espera_veh	0.8	0.8	0.8	0.8
w_espera_peat	0.6	0.6	0.8	0.8
w_ocupacion_aguas_abajo	0.4	0.4	0.4	0.4
umbral_spillback_pct	0.85	0.85	0.85	0.85
I1: ciclo/vp/vs/vpeat/desfase (s)	95 / 44 / 28 / 16 / 0	57 / 18 / 12 / 12 / 0	57 / 18 / 12 / 12 / 0	57 / 18 / 12 / 12 / 0
I2: ciclo/vp/vs/vpeat/desfase (s)	85 / 40 / 24 / 14 / 22	57 / 16 / 10 / 16 / 18	57 / 16 / 10 / 16 / 18	57 / 16 / 10 / 16 / 18
I3: ciclo/vp/vs/vpeat/desfase (s)	85 / 46 / 22 / 11 / 44	57 / 13 / 10 / 19 / 30	57 / 13 / 10 / 19 / 30	57 / 13 / 10 / 19 / 30
I4: ciclo/vp/vs/vpeat/desfase (s)	95 / 50 / 26 / 12 / 61	57 / 18 / 12 / 12 / 51	57 / 18 / 12 / 12 / 51	57 / 18 / 12 / 12 / 51

El Ingeniero partió de `propuesta_base` (ciclo común de 57 s, verdes y desfases calculados por `calculadora_webster`). El guardrail bloqueó 1 intento(s) del modelo durante la corrida y el propio modelo se corrigió al recibir el error; el código no tuvo que fijar nada. El peso `w_espera_peat` quedó en 0.8 (vigente: 0.6). Se publicó la propuesta_v2.

Tarea 5 — Auditoría del ciclo 2 y publicación.


```
{
  "ciclo_de_auditoria": "Auditoría de propuesta corregida",
  "dictamen_propuesta": {
    "veredicto": "APROBADO",
    "violaciones": [],
    "advertencias": []
  },
  "reglas_evaluadas": [
    "R0 esquema y completitud",
    "R1 verde vehicular",
```

```
    "R2 ciclo maximo",
    "R3 despeje peatonal",
    "R4 consistencia de fases",
    "R5 espera peatonal maxima",
    "R6 espera maxima por carril"
  ],
  "autorizacion_de_publicacion": true,
  "publicacion": {
    "resultado": "publicado",
    "archivo": "configuracion_aprobada.json",
    "fuente": "propuesta_v2"
  }
}
```

Respuesta final consolidada

Reporte del Sintetizador, con la verificación determinista que `verificacion.py` le agregó (se reproduce el contenido sustantivo; el archivo completo es `reporte_ciclo_supervision.md`):

```
> **Verificacion automatica:** el Auditor publico una configuracion aprobada (`configuracion_aprobada.json`).


# Reporte del Ciclo de Supervisión para Coordinación Operativa del Campus


## Introducción
Este reporte presenta el análisis del desempeño de SIGMA-Vial durante el escenario pico matutino, las dos propuestas y
  los dos dictámenes. Se explican los hallazgos, la justificación para la rechazo de la primera propuesta, los cambios
  realizados y el costo asumido. Además, se incluye una sección de trazabilidad que detalla de qué archivo salió cada
  elemento.


## Hallazgos Principales
- **Intersección I1**: Mejora significativa en espera peatonal y spillback.
- **Intersección I2**: Empeoramiento significativo en spillback.
- **Intersección I3**: Empeoramiento significativo en espera peatonal y spillback.
- **Intersección I4**: Mejora en espera peatonal y spillback.


### Tabla Comparativa de Parámetros


| Intersección | Espera Peatonal (vs. Plan Fijo Webster) | Espera Peatonal (vs. Máxima Presión) | Spillback (vs. Plan
  Fijo Webster) | Spillback (vs. Máxima Presión) |
|--------------|-----------------------------------------|--------------------------------------|-------------------------
  --------|------------------------------|
| I1           | -37.4% (p < 0.05, MEJORA)               | -46.4% (p < 0.05, MEJORA)            | -81.4% (p < 0.05,
  MEJORA)        | 6.3% (p > 0.05, SIN DIFERENCIA SIGNIFICATIVA) |
| I2           | -26.9% (p < 0.05, MEJORA)               | -47.1% (p < 0.05, MEJORA)            | 105.6% (p > 0.05, SIN
  DIFERENCIA SIGNIFICATIVA) | 589.2% (p < 0.05, EMPEORA) |
| I3           | 64.3% (p < 0.05, EMPEORA)               | 26.6% (p < 0.05, EMPEORA)            | 96.8% (p > 0.05, SIN
  DIFERENCIA SIGNIFICATIVA) | 29.8% (p > 0.05, SIN DIFERENCIA SIGNIFICATIVA) |
| I4           | -29.3% (p < 0.05, MEJORA)               | -38.1% (p < 0.05, MEJORA)            | -76.8% (p < 0.05,
  MEJORA)        | -5.3% (p > 0.05, SIN DIFERENCIA SIGNIFICATIVA) |


## Justificación para la Rechazo de la Primera Propuesta
La primera propuesta fue rechazada debido a que violaba varias reglas duras establecidas por la normativa de cruce
  peatonal. Las violaciones incluyen:
- Exceso de tiempo de ciclo en varias intersecciones.
- Espera peatonal máxima implícita excediendo el límite permitido.
- Espera máxima por carril excediendo el límite permitido.


## Cambios Realizados y Costo Asumido
- **Intersección I3**: El verde peatonal se ajustó a 19 segundos para cumplir con la regla R3 (despeje peatonal) para un
  cruce de 14.0m a 1.0 m/s.
- **Intersección I1, I2, I4**: Se mantuvieron los ciclos en 57 segundos para cumplir con la regla R4 (ciclo máximo
  permitido).


### Costo Asumido
- **Intersección I3**: Se sacrifica un despeje peatonal de 14.0s a 19.0s, lo que podría afectar la fluidez peatonal en
  el cruce.
- **Intersección I1, I2, I4**: Se mantiene el ciclo en 57 segundos, lo que podría afectar la fluidez vehicular en estos
  puntos.


## Trazabilidad
- **Hallazgos**: Se obtuvieron de la tabla de hallazgos en el archivo `hallazgos_pico_matutino.json`.
- **Parámetros de la Propuesta**: Se obtuvieron de la sección `planes_por_interseccion` en el archivo
  `propuesta_v2.json`.
- **Dictámenes**: Se obtuvieron de los archivos `dictamen_configuracion_vigente.json` y `dictamen_propuesta.json`.
- **Configuración Aprobada**: Publicada en el archivo `configuracion_aprobada.json`.
```

```
## Conclusión
Las intersecciones I3 y I2 presentan problemas significativos en el pico matutino, con mayor demanda de peatones y
  eventos de spillback. Se recomienda revisar la configuración o ajustar la demanda en estas intersecciones para mejorar
  el desempeño.


---


Este reporte proporciona una visión clara de los hallazgos, la justificación para la rechazo de la primera propuesta,
  los cambios realizados y el costo asumido, con el fin de garantizar que la coordinación operativa del campus tenga una
  comprensión clara de la situación y las decisiones tomadas.


---
## Tabla de parametros antes y despues (generada por codigo)


| Parametro | v1.0-fase2 | v1.1 |
|---|---|---|
| peso w_espera_peat | 0.6 | 0.8 |
| peso w_espera_veh | 0.8 | 0.8 |
| peso w_ocupacion_aguas_abajo | 0.4 | 0.4 |
| peso w_peat_atendidos | 1.0 | 1.0 |
| peso w_veh_atendidos | 1.0 | 1.0 |
| umbral_spillback_pct | 0.85 | 0.85 |
| periodo_decision_s | 5 | 5 |
| I1 ciclo_s | 95 | 57 |
| I1 verde_principal_s | 44 | 18 |
| I1 verde_secundario_s | 28 | 12 |
| I1 verde_peatonal_s | 16 | 12 |
| I1 desfase_s | 0 | 0 |
| I2 ciclo_s | 85 | 57 |
| I2 verde_principal_s | 40 | 16 |
| I2 verde_secundario_s | 24 | 10 |
| I2 verde_peatonal_s | 14 | 16 |
| I2 desfase_s | 22 | 18 |
| I3 ciclo_s | 85 | 57 |
| I3 verde_principal_s | 46 | 13 |
| I3 verde_secundario_s | 22 | 10 |
| I3 verde_peatonal_s | 11 | 19 |
| I3 desfase_s | 44 | 30 |
| I4 ciclo_s | 95 | 57 |
| I4 verde_principal_s | 50 | 18 |
| I4 verde_secundario_s | 26 | 12 |
| I4 verde_peatonal_s | 12 | 12 |
| I4 desfase_s | 61 | 51 |



---
**Verificacion automatica de cifras.** Todas las cifras del reporte aparecen en las salidas de las tareas anteriores o
  en los datos.


**Verificacion automatica de archivos.** El reporte cita archivos que no existen en el proyecto:
  `dictamen_configuracion_vigente.json`, `dictamen_propuesta.json`, `hallazgos_pico_matutino.json`. La trazabilidad de
  esos elementos no es confiable.
```

D.8 Qué demuestra la corrida y qué no

Lo que sí demuestra:

- **El Auditor tiene poder real de veto.** El validador rechazó la configuración vigente (13 violaciones de R4, R5, R6 y R3) y el modelo reportó ese veredicto en su dictamen. El veredicto lo emite Python, no el modelo.
- **La doble barrera funciona.** El publicador revalida antes de escribir; en las corridas de prueba bloqueó publicaciones inválidas y nunca escribió una configuración que no pasara R0-R6.
- **Las restricciones se imponen por código y el modelo se corrige.** El guardrail bloqueó 1 intento(s) y la tarea terminó sin intervención del código.
- **La verificación del reporte detecta invenciones.** Las cifras del reporte aparecen todas en los datos. Además señaló archivos inexistentes que el Sintetizador citó en su trazabilidad: `dictamen_configuracion_vigente.json`, `dictamen_propuesta.json`, `hallazgos_pico_matutino.json`.

Lo que no demuestra:

- Que SIGMA-Vial mejore el circuito real: la telemetría es sintética y las propuestas no se evalúan en desempeño, solo en cumplimiento de reglas.
- Que el LLM diagnostique mejor que un ingeniero de tránsito: no se comparó. Un 7B local es inconsistente (D.9).
- Que la capa de tiempo real consuma la configuración publicada: esa integración está pendiente.

D.9 Corridas de prueba y hallazgos

Antes de la corrida de demostración se ejecutó el Crew varias veces con distintas versiones del código y del modelo. Se reportan todas, incluidas las que fallaron, porque cada una motivó un cambio de diseño.

#	Versión del código	Modelo	Resultado y lección
1	Primera versión: sin esquema de salida, delegación del Ingeniero activa	llama3.1:8b	34 min. El Ingeniero respondió prosa en lugar de JSON; la propuesta v2 contenía el dictamen de otra tarea; no publicó. El Sintetizador inventó un «costo de $10,000» y archivos. → Se descartó el modelo, se agregó salida estructurada y se apagó la delegación.
2	+ `output_pydantic`	qwen2.5:7b	13.7 min. JSON válido, pero ciclos de 120-130 s (viola R2); no publicó; el reporte afirmó «cumpliendo con las reglas duras». → Calculadora de Webster restringida y verificación del reporte.
3	+ calculadora restringida	qwen2.5:7b	≈3 min; **publicó, pero la configuración tenía solo 2 de las 4 intersecciones** y umbral de spillback 10.0 (el esquema usa fracciones). → Regla R0 (esquema y completitud) y esquema estricto.
4	+ R0, validación por archivo	qwen2.5:7b	24 min, sin publicar: la v1 era válida, pero el Ingeniero la «corrigió» en v2 y la invalidó. → Publicación determinista `ultima_valida`.
5	idem	qwen2.5:7b	**Bucle de generación**: 3 llamadas consecutivas de 10 min con error 500; la corrida se detuvo a mano. → `max_tokens=3000` y `timeout=240 s`.
6	+ calculadora con propuesta base (5 corridas)	qwen2.5:7b	2 de 5 publican. En las otras 3 el modelo alteró los planes. → Guardrail.
7	+ guardrail (5 corridas)	qwen2.5:7b	5 de 5 publican. Pero el diagnóstico de una corrida pasó por alto que I3 empeora. → Lectura explícita MEJORA/EMPEORA en el comparador.
8	+ lectura explícita (3 corridas)	qwen2.5:7b	3 de 3 publican; el modelo sube el peso peatonal. Pero en una corrida el Auditor reescribió a mano el JSON que debía validar, incompleto, y el validador rechazó un archivo distinto del que produjo el Ingeniero. → El LLM solo puede indicar el **nombre del archivo** a validar o publicar.
9	+ solo nombre de archivo (3 corridas)	qwen2.5:7b	2 de 3 publican; en la otra, el modelo devolvió una respuesta vacía y CrewAI abortó toda la corrida. → Reintento automático (hasta 3 veces).
10	+ reintento automático (3 corridas) y corrida de demostración del notebook	qwen2.5:7b	3 de 3 publican; el notebook final también publicó. **Versión definitiva.**

Resultados por serie de ejecuciones con el modelo qwen2.5:7b (dentro de cada serie, mismo código y mismos datos):

Serie	Corridas	Publicaron	Duración media	Errores 500
Sin guardrail	5	2 (40 %)	7.9 min	0
Con guardrail	5	5 (100 %)	9.2 min	0
Con guardrail + lectura explicita en el comparador	3	3 (100 %)	3.8 min	0
Version definitiva, sin reintento automatico	3	2 (66 %)	5.1 min	1
Version definitiva, con reintento automatico	3	3 (100 %)	5.1 min	0

En palabras simples: cada serie es un grupo de ejecuciones completas del equipo con el mismo modelo y los mismos datos; como el modelo no responde exactamente igual cada vez, se repite para ver qué tan seguido sale bien. El **guardrail** fue la mejora de mayor efecto: sin él, el modelo alteraba los tiempos de semáforo pese a que se le pedía copiarlos y la propuesta quedaba inválida en 3 de 5 ejecuciones. Con él, el guardrail bloqueó el primer intento en 4 de las 5 ejecuciones y el modelo se corrigió al recibir el error con los valores exactos. La segunda mejora fue decir en palabras MEJORA o EMPEORA en la salida del comparador: antes, en una ejecución, el modelo concluyó «mejora significativa» en las cuatro intersecciones aunque en I3 la espera peatonal empeora 64.3 %. Desde ese cambio (9 ejecuciones en las tres últimas series), 8 publicaron una configuración; en 7 de esas 8 el modelo subió el peso de la espera peatonal respecto al vigente (0.6), y en 5 de las 7 ejecuciones cuyo diagnóstico pudo evaluarse puso a I3 como primera prioridad (en el resto puso a I2). Es decir: **el diagnóstico mejoró, pero sigue siendo irregular.** En la serie «sin reintento», una ejecución se cayó porque el modelo devolvió una respuesta vacía; por eso se agregó el reintento automático. **Las muestras son pequeñas (3 a 5 ejecuciones por serie):** muestran una tendencia, no una tasa confiable.

D.10 Limitaciones declaradas

- **La telemetría es sintética.** El circuito en SUMO no está construido y los aforos de campo están pendientes. Los valores absolutos no son medidas del circuito real.
- **Los hallazgos de la corrida están sembrados.** El desajuste de I3 y la configuración vigente son datos de prueba; no son defectos de las Fases 1-2. Solo la derivación analítica del ciclo máximo factible (B.5) es un resultado de diseño sobre la especificación.
- **Un modelo local de 7B es inconsistente.** Entre corridas idénticas, el resultado varía; el reporte del Sintetizador puede contener afirmaciones cualitativas incorrectas o cifras copiadas fuera de contexto que la verificación numérica no detecta.
- **Parte del comportamiento correcto lo impone el código, no el modelo:** los planes los calcula una herramienta, un guardrail los fija si el modelo los altera y la publicación la decide una regla determinista. El LLM aporta el diagnóstico, los pesos de utilidad y la redacción.
- **No hay evaluación de desempeño de las propuestas.** Falta una herramienta que simule la configuración propuesta en SUMO y devuelva métricas.
- **No hay integración con la capa de tiempo real.** El Crew escribe `configuracion_aprobada.json`; su consumo por el Controlador está pendiente.
- **El lazo de corrección tiene exactamente dos pasadas.** Si ninguna propuesta es válida, el flujo termina sin publicar y la configuración vigente sigue ejecutándose; conviene registrar esa condición como alarma y definir un plan seguro de respaldo.
- **Parámetros normativos sin verificar.** Ámbar, todo-rojo, velocidad peatonal y esperas máximas son los de la ficha, marcados allí como «[Confirmar con normativa local]».

- **Supuestos de la calculadora de planes:** fase peatonal exclusiva una vez por ciclo; flujo 60/40 entre acceso principal y secundario; grado de saturación objetivo de 0.9; desfases con 7 m por vehículo de capacidad a 30 km/h.

Conclusiones y trabajo siguiente

El rediseño de SIGMA-Vial hacia una arquitectura multiagente obligó a una distinción que no estaba en las fases anteriores: no todo el problema admite agentes basados en modelos de lenguaje. El lazo de control del semáforo exige latencia acotada y verificabilidad determinista, y ahí los agentes LLM son la herramienta equivocada. La supervisión táctica (interpretar por qué el sistema se degradó, cruzar el diagnóstico con la normativa, ponderar objetivos en conflicto y justificar un cambio) es una capa distinta, con otra cadencia, y es donde la arquitectura MAS aporta valor.

La implementación con un modelo local de 7B dejó una lección más útil que la configuración publicada: **un LLM pequeño no debe decidir nada verificable**. Cada falla observada (propuestas con planes inválidos, una configuración incompleta que pasó la auditoría, un reporte que inventaba costos y archivos, un bucle de generación) se resolvió llevando esa decisión a código determinista: reglas R0-R6, una calculadora de planes factibles, un guardrail, una regla de publicación y una verificación del reporte. El valor del MAS está en el diagnóstico, la ponderación y la explicación, no en el cálculo ni en la validación.

Además, el diseño de las reglas sugiere una revisión de la especificación de las Fases 1-2: con fases exclusivas y esperas máximas de 60 s, el ciclo máximo factible es de unos 82 s, no de 120 s (B.5).

Trabajo siguiente

Prioridad	Acción
Inmediata	Revisar la especificación de las Fases 1-2: reconciliar el ciclo máximo de 120 s con las esperas máximas de 60 s (fases peatonales concurrentes, doble servicio por ciclo, o relajar el límite con justificación) y confirmar los parámetros con la normativa local.
Inmediata	Levantar el aforo vehicular y peatonal en campo y sustituir los nombres y volúmenes provisionales de las cuatro intersecciones.
Fase 4	Construir el circuito en SUMO desde OpenStreetMap, calibrar la demanda y sustituir la telemetría sintética por la real.
Fase 4	Agregar la herramienta `simular_configuracion` (SUMO/TraCI) para evaluar el desempeño de cada propuesta antes de auditarla, y optimización paramétrica como referencia.
Fase 4	Implementar el consumo de `configuracion_aprobada.json` por el Controlador de la capa de tiempo real.
Fase 5	Ejecutar el Crew con un LLM de mayor capacidad sobre los cuatro escenarios y comparar la calidad de sus diagnósticos con los de un ingeniero de tránsito.

Referencias

Bazzan, A. L. C. (2009). Opportunities for multiagent systems and multiagent reinforcement learning in traffic control. *Autonomous Agents and Multi-Agent Systems, 18*(3), 342–375. https://doi.org/10.1007/s10458-008-9062-9

CrewAI Inc. (2026). *CrewAI documentation: Agents, tasks, crews and tools*. https://docs.crewai.com

Foundation for Intelligent Physical Agents. (2002). *FIPA ACL message structure specification* (Documento SC00061G). FIPA.

Grattafiori, A., Dubey, A., Jauhri, A., Pandey, A., Kadian, A., Al-Dahle, A., … Ma, Z. (2024). *The Llama 3 herd of models* (arXiv:2407.21783). arXiv. https://doi.org/10.48550/arXiv.2407.21783

Huang, J., Chen, X., Mishra, S., Zheng, H. S., Yu, A. W., Song, X., & Zhou, D. (2024). Large language models cannot self-correct reasoning yet. En *The Twelfth International Conference on Learning Representations (ICLR 2024)*. https://doi.org/10.48550/arXiv.2310.01798

Hunt, P. B., Robertson, D. I., Bretherton, R. D., & Winton, R. I. (1981). *SCOOT: A traffic responsive method of coordinating signals* (TRRL Laboratory Report 1014). Transport and Road Research Laboratory.

Lopez, P. A., Behrisch, M., Bieker-Walz, L., Erdmann, J., Flötteröd, Y.-P., Hilbrich, R., Lücken, L., Rummel, J., Wagner, P., & Wießner, E. (2018). Microscopic traffic simulation using SUMO. En *2018 21st International Conference on Intelligent Transportation Systems (ITSC)* (pp. 2575–2582). IEEE. https://doi.org/10.1109/ITSC.2018.8569938

Ollama. (2026). *Ollama* [Software]. https://ollama.com

Qwen Team. (2024). *Qwen2.5 technical report* (arXiv:2412.15115). arXiv. https://doi.org/10.48550/arXiv.2412.15115

Russell, S., & Norvig, P. (2021). *Artificial intelligence: A modern approach* (4th ed.). Pearson.

Secretaría de Desarrollo Agrario, Territorial y Urbano. (2019). *Manual de calles: Diseño vial para ciudades mexicanas*. SEDATU; Banco Interamericano de Desarrollo.

Sims, A. G., & Dobinson, K. W. (1980). The Sydney coordinated adaptive traffic (SCAT) system philosophy and benefits. *IEEE Transactions on Vehicular Technology, 29*(2), 130–137. https://doi.org/10.1109/T-VT.1980.23833

Varaiya, P. (2013). Max pressure control of a network of signalized intersections. *Transportation Research Part C: Emerging Technologies, 36*, 177–195. https://doi.org/10.1016/j.trc.2013.08.014

Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., Cournapeau, D., Burovski, E., Peterson, P., Weckesser, W., Bright, J., van der Walt, S. J., Brett, M., Wilson, J., Millman, K. J., Mayorov, N., Nelson, A. R. J., Jones, E., Kern, R., Larson, E., … SciPy 1.0 Contributors. (2020). SciPy 1.0: Fundamental algorithms for scientific computing in Python. *Nature Methods, 17*(3), 261–272. https://doi.org/10.1038/s41592-019-0686-2

Webster, F. V. (1958). *Traffic signal settings* (Road Research Technical Paper No. 39). Her Majesty's Stationery Office.

Wei, H., Zheng, G., Gayah, V., & Li, Z. (2019). *A survey on traffic signal control methods* (arXiv:1904.08117). arXiv. https://doi.org/10.48550/arXiv.1904.08117

Wooldridge, M. (2009). *An introduction to multiagent systems* (2nd ed.). John Wiley & Sons.

Anexo. Acceso al código fuente y reproducción

Entrega	**Código adjunto en** `SIGMA_Vial_Fase3_entrega.zip`**. [Pendiente: insertar enlace a GitHub o Google Colab si el equipo lo publica]**
Archivos	`sigma_crew.py`, `herramientas.py`, `verificacion.py`, `generar_telemetria.py`, `llm_stub.py`, `SIGMA_Vial_Fase3.ipynb`, `Modelfile.qwen25`, `requirements.txt`, `README.md`, carpeta `datos/` y `salidas/`, `reporte_ciclo_supervision.md`, `corrida_demostracion.log`.
Entorno	Python 3.12 (CrewAI no soporta 3.14). CrewAI 1.15.35, pandas 3.0.5, scipy 1.18.1, pydantic 2.12.5. Versiones fijadas en `requirements.txt`.
Reproducir (sin modelo)	`pip install -r requirements.txt` · `python generar_telemetria.py` · `python sigma_crew.py` (modo stub: solo para depurar; no demuestra razonamiento).
Con razonamiento real	Instalar Ollama · `ollama pull qwen2.5:7b` · `ollama create sigma-qwen25 -f Modelfile.qwen25` · `SIGMA_LLM=ollama python sigma_crew.py`. En PowerShell: `$env:SIGMA_LLM="ollama"`.
Modelo de la corrida	`qwen2.5:7b` (Qwen Team, 2024), cuantización Q4_K_M, contexto 16 384 tokens, vía Ollama; GPU RTX 5050 de 8 GB.